

Understanding Prompts in AI

System • User • Assistant

A plain-English guide with Python examples

Introduction

When you talk to an AI like Claude or ChatGPT, the conversation is made up of three different types of messages. Each type has a specific role — like actors in a play. Understanding these three roles helps you get much better results from any AI model.

The three roles are:

- **System Prompt** — Sets the rules and personality of the AI.
- **User Prompt** — The message or question you (the human) send.
- **Assistant Prompt** — The AI's reply.

Think of it like ordering at a restaurant. The **system prompt** is the restaurant's training manual that tells the waiter how to behave. The **user prompt** is your order. The **assistant prompt** is what the waiter brings back to your table.

1. System Prompt

The system prompt is a hidden set of instructions written by the developer or app builder. It tells the AI *who it is*, *how to behave*, and *what rules to follow* — before the user even types a word.

Key characteristics:

- Set once at the beginning of a session.
- Usually invisible to the end user.
- Defines the AI's persona, tone, and boundaries.
- Written by the developer, not the user.

Example — Basic System Prompt in Python

```
import anthropic

client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-opus-4-6",
    max_tokens=512,
    system=( # <-- This is the SYSTEM PROMPT
        "You are a friendly cooking assistant. "
        "Always suggest simple recipes. "
        "Never recommend recipes that take more than 30 minutes."
    ),
    messages=[
        {"role": "user", "content": "What can I make with eggs and cheese?"}
    ],
)
print(response.content[0].text)
```

In the example above, the system prompt quietly shapes every response. The AI will always behave like a cooking assistant — even if the user asks something off-topic.

Tip: A good system prompt is specific. Instead of 'Be helpful', write 'You are a customer support agent for Acme Corp. Only answer questions about our products.'

2. User Prompt

The user prompt is the message that the human sends to the AI. This is the most familiar part — it's what you type into a chatbot. It can be a question, a command, a piece of text to analyse, or anything else you want the AI to work on.

Key characteristics:

- Sent by the human in the conversation.
- Can change with every turn.
- Visible to both the user and the AI.
- Drives what the AI actually does in each response.

Example — Sending a User Prompt in Python

```
import anthropic

client = anthropic.Anthropic()

# The user prompt is inside the 'messages' list with role='user'
response = client.messages.create(
    model="claude-opus-4-6",
    max_tokens=512,
    messages=[
        {
            "role": "user",    # <-- USER PROMPT
            "content": "Explain photosynthesis in simple words."
        }
    ],
)
print(response.content[0].text)
```

How to write a good user prompt:

The quality of the AI's answer depends heavily on how clearly you ask. Here are simple tips:

Tip	Example
Be specific	Instead of 'Tell me about Python', say 'Give me 3 use cases of Python in data science.'
Give context	Instead of 'Fix this', say 'Fix this Python function that should sort a list.'
State the format	Add 'Answer in bullet points' or 'Keep it under 100 words.'

One task at a time

Avoid asking five things in one message. Break them up.

3. Assistant Prompt

The assistant prompt is the AI's reply. In most everyday use, you just read it. But in advanced setups — especially when building AI apps — you can *pre-fill* the assistant's response to steer the AI, create personas, or continue a previous answer.

Key characteristics:

- Generated by the AI model.
- Can be pre-filled by the developer to guide the AI's next reply.
- Visible in the conversation history.
- Used to build multi-turn (back-and-forth) conversations.

Example — Reading the Assistant's Response

```
import anthropic

client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-opus-4-6",
    max_tokens=256,
    messages=[
        {"role": "user", "content": "What is 2 + 2?"}
    ],
)

# The assistant prompt is in the response
assistant_reply = response.content[0].text # <-- ASSISTANT PROMPT
print(assistant_reply) # Output: 4
```

Advanced — Pre-filling the Assistant's Response

You can add an assistant message with partial content to force the AI to continue from that point. This is called **response pre-filling**.

```
import anthropic

client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-opus-4-6",
    max_tokens=300,
    messages=[
        {
            "role": "user",
            "content": "List the planets in our solar system."
        },
        {
            "role": "assistant", # <-- PRE-FILLED ASSISTANT PROMPT
            "content": "Here are the 8 planets:" # AI continues from here
        },
    ],
)
print(response.content[0].text)
# The AI will continue the list without re-stating the intro line.
```

Warning: Pre-filling assistant responses is a powerful technique but use it carefully. If you pre-fill incorrect information, the AI may continue building on that wrong foundation.

4. All Three Together — A Full Example

Here is how a real multi-turn conversation uses all three prompt types together. Imagine building a simple customer support chatbot for a software company.

[illegible]

What happens in each turn:

SYSTEM	You are a friendly support agent for TechCorp. Only answer questions about TechCorp software.
USER	How do I reset my password?
ASSISTANT	Sure! To reset your password, go to the login page and click 'Forgot Password'. You will receive an email with a reset link.
USER	What are your office hours?
ASSISTANT	I can only help with TechCorp software questions. For general enquiries, please visit the TechCorp website. Is there anything software-related I can help you with?

4b. All Three Together — With Response Pre-filling

The previous example shows a normal multi-turn chat. Now let's add **response pre-filling**: we supply the beginning of the assistant's reply ourselves, forcing the AI to continue from exactly that point. This is useful when you want the AI to always answer in a fixed format — for example, a structured troubleshooting guide.

Scenario: A TechCorp support bot that always responds to bug reports with a numbered step-by-step diagnosis, starting with a pre-filled opening line.

[illegible]

How each prompt type contributes:

SYSTEM	You are a technical support agent for TechCorp. When diagnosing a bug, always list steps as: Step 1, Step 2, Step 3 ... and end with a summary.
USER	The app crashes every time I click the Export button.
ASSISTANT (pre-filled)	Got it! Here is how we will diagnose this crash: ← We wrote this. The AI continues from here.
ASSISTANT (AI continues)	Step 1: Update the app ... Step 2: Clear the cache ... Summary: If steps 1-3 do not fix it, share the logs with us.

Why is this useful? Without pre-filling, the AI might start with 'Sure, I can help!' or 'That sounds frustrating.' With pre-filling, you guarantee the reply always opens with the exact line you want — keeping responses consistent across thousands of users.

5. Quick Reference — Side by Side

Feature	System Prompt	User Prompt	Assistant Prompt
Who writes it?	Developer / App	End user	The AI model
When is it set?	Before conversation	Each message	AI's reply
Purpose	Set rules & persona	Ask questions/tasks	Give answers
Visible to user?	Usually hidden	Yes	Yes
Can be changed?	By developer only	Yes, each turn	By AI logic
Example	'You are a helpful chef'	'How do I boil pasta?'	'Boil water first...'

6. Common Mistakes to Avoid

X Vague system prompts

A system prompt like 'Be helpful' gives the AI almost no direction. Be specific: 'You are a tax advisor. Only discuss UK tax laws. Always recommend consulting a certified accountant.'

X Putting system instructions in the user prompt

Some beginners write 'Act like a pirate and answer my question' inside the user message every time. This is inefficient. Put persona rules in the system prompt instead.

X Forgetting conversation history

AI models have no memory by default. If you want the AI to remember earlier messages, you must pass the full conversation list with every API call.

X Over-stuffing the user prompt

Asking 10 things in one message leads to incomplete answers. Break complex tasks into separate, focused messages.

X Ignoring the assistant's context

In multi-turn chats, the AI sees all previous assistant replies. If an earlier reply was wrong, explicitly tell the AI to correct it.

7. Best Practices

1. Write your system prompt first	Before building a chatbot, define the system prompt clearly. Think: Who is the AI? What can it do? What must it never do?
2. Keep user prompts focused	One clear question per turn gets a much better answer than a long, multi-part request.
3. Use examples in prompts	Show the AI what a good answer looks like. E.g., 'Answer in this format: Step 1: ... Step 2: ... Step 3: ...'
4. Test with edge cases	Try asking your chatbot off-topic or tricky questions to see if the system prompt holds up.
5. Store conversation history properly	Build a list that grows with each turn. Always pass the full list to the API.
6. Iterate and improve	Prompt engineering is not a one-shot task. Test, find where the AI goes wrong, and refine your prompts.

Mastering these three prompt types — System, User, and Assistant — is the foundation of working effectively with any AI model. Once you understand their roles, you can build smarter chatbots, get better answers, and design AI applications that truly do what you want.

Happy prompting!